

## Algoritmo Backpropagation – Treinamento de Redes Neurais

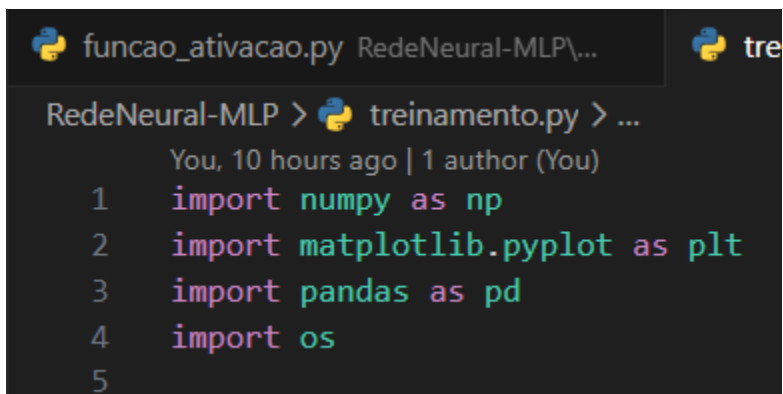
### Bibliotecas utilizadas:

**NumPy:** Responsável por todas as operações matemáticas estruturadas, criação e manipulação das matrizes de pesos e vetores de entrada e saída.

**Matplotlib:** Utilizada na geração de gráficos de desempenho para acompanhar a relação Erro × Época ao longo do tempo.

**Pandas:** Aplicada na exportação dos resultados gerados nos testes para o formato CSV e no registro detalhado dos experimentos

**OS** Armazenamento e Diretório de pastas



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar displays the file name 'funcao\_ativacao.py' and the directory 'RedeNeural-MLP\...'. The main area shows the code for 'treinamento.py' with the following imports:

```
RedeNeural-MLP > import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
```

Line numbers 1 through 5 are visible on the left side of the code block.

## Slide 2 – Funções de Ativação (Sigmóide)

### Implementação da Função Sigmóide Binária clássica:

Transforma qualquer valor numérico real recebido em um intervalo restrito **entre 0 e 1**.

**Fundamental** na arquitetura da rede para introduzir a não linearidade, permitindo que o modelo aprenda padrões complexos e não separáveis linearmente.

```
RedeNeural-MLP > src > 🐍 funcao_ativacao.py > ...  
You, 6 days ago | 1 author (You)  
1  import numpy as np      You, last week • inicial  
2  
3  # Calcula a função de ativação sigmoide binária  
4  def sigmoide_binaria(x):  
5  
6  # Retorna valores no intervalo de 0 a 1  
7  |     return 1 / (1 + np.exp(-x))  
8  
9  # Calcula a derivada da sigmoide binária  
10 # uso no Backpropagation  
11 def derivada_sigmoide_binaria(x):
```

### Slide 3 – Funções de Ativação (Bipolar e TanH)

**Sigmóide Bipolar:** Mapeia as saídas na faixa de variação de **-1 até 1**.

**Tangente Hiperbólica (np.tanh):** Função nativa que também opera na faixa de **-1 até 1**, oferecendo uma resposta de gradiente diferente durante o ajuste de pesos.

```
38 # Calcula a função tangente hiperbólica (tanh)
39 def tangente_hiperbolica(x):
40
41     # Retorna valores no intervalo de -1 a 1
42     return np.tanh(x)
43
44
45 # Calcula a derivada da tangente hiperbólica
46 # faz a retropropagação do erro
47 def derivada_tangente_hiperbolica(x):
48
49     # Retorna valores no intervalo de -1 a 1
50     return 1 - np.tanh(x) ** 2
```

## Slide 4 – Visualização das Funções de Ativação

Estruturação visual dos comportamentos das funções:

**np.linspace(-10, 10, 1000)**: Gera 1000 pontos linearmente espaçados no intervalo de -10 a 10 para criar curvas.

**plt.plot(x, sigmoid(x))**: Executa a plotagem do gráfico cartesiano da função sigmóide para validação visual das faixas de transição.

```
RedeNeural-MLP >  graficos.py > ...  
10  
11 # cria valores de x entre -10 e 10  
12 # desenhar as curvas  
13 x = np.linspace(-10, 10, 1000)  
14  
15  
16 You, last week • Implementa func
```

## Slide 5 – Arquitetura da Rede Neural MLP

### Configuração e instanciação do objeto da rede MultiLayer Perceptron

**2 neurônios** na camada de entrada (adequados para as duas variáveis lógicas do XOR).

**4 neurônios** na camada intermediária/oculta.

**1 neurônio** na camada de saída (resultado final).

**Taxa de aprendizagem** inicial fixada em 0,2 para guiar o gradiente.

```
RedeNeural-MLP > src > mlp.py > ...  
You, 10 hours ago | 1 author (You)  
1 import numpy as np  
You, 10 hours ago | 1 author (You)  
2 class mlp:  
3  
4     # construtor da rede  
5     def __init__(  
6  
7         self,  
8  
9         entradas=2,  
10  
11         ocultos=4,  
12  
13         saidas=1,  
14  
15         taxa=0.2  
16     ):
```

## Slide 6 – Inicialização dos Pesos

### Definição dos parâmetros iniciais da estrutura sináptica

**np.random.uniform(-1, 1, (entradas, ocultos)):** Cria uma distribuição aleatória uniforme de pesos contidos entre -1 e 1.

**Objetivo:** Evitar a simetria matemática e impedir que todos os neurônios da camada oculta aprendam exatamente a mesma coisa.

```
# salva taxa de aprendizagem
self.taxa = taxa

# cria pesos entre entrada e camada oculta
self.pesos_entrada_oculta = np.random.uniform(-1, 1, (entradas, ocultos))

# cria pesos entre camada oculta e saída
self.pesos_oculta_saida = np.random.uniform(-1, 1, (ocultos, saidas))

# cria bias da camada oculta
self.bias_oculta = np.random.uniform(-1, 1, (1, ocultos))

# cria bias da camada de saída
self.bias_saida = np.random.uniform(-1, 1, (1, saidas))
```

## Slide 7 – Etapa Forward (Propagação Direta)

### Mecanismo de processamento de sinais dentro da rede

**self.soma\_oculta:** Realiza o produto escalar (`np.dot`) entre as entradas `X` e a matriz de pesos, somando ao vetor de bias para encontrar a entrada líquida.

**self.saida\_oculta:** Aplica a função de ativação sobre o resultado líquido obtido.

**self.saida\_final:** Repete o cálculo para a camada seguinte, produzindo a saída final predita pelo modelo.

```
# etapa FORWARD
def forward(self, X):

    # calcula entrada da camada oculta
    self.soma_oculta = (np.dot(X, self.pesos_entrada_oculta) + self.bias_oculta)

    # ativa camada oculta
    self.saida_oculta = self.sigmoid(self.soma_oculta)

    # calcula entrada da camada de saída
    self.soma_saida = (np.dot(self.saida_oculta, self.pesos_oculta_saida) + self.bias_saida)

    # ativa saída final
    self.saida_final = self.sigmoid(self.soma_saida)

    # retorna resultado
    return self.saida_final
```

## Slide 8 – Etapa Backpropagation (Retropropagação do Erro)

### Cálculo dos gradientes de correção de falhas

**erro\_saida:** Determina a distância entre o gabarito esperado (**y**) e a saída real calculada.

**delta\_saida:** Multiplica o erro pela derivada da função de ativação, calculando a contribuição exata de cada peso na falha final.

**erro\_oculta:** Propaga o vetor de erro de volta (para trás) multiplicando o delta pela transposta (**.T**) da matriz de pesos.

```
# etapa BACKWARD
def backward(self, X, y):    You, last week • Implementacao da rede mlp

    # calcula erro da saída
    erro_saida = (y - self.saida_final)

    # calcula delta da saída
    delta_saida = (erro_saida * self.derivada_sigmoid(self.saida_final))

    # propaga erro para camada oculta
    erro_oculta = np.dot(delta_saida, self.pesos_oculta_saida.T)

    # calcula delta da camada oculta
    delta_oculta = (erro_oculta * self.derivada_sigmoid(self.saida_oculta))
```

## Slide 9 – Atualização dos Pesos Sinápticos

### Aplicação prática da regra delta para o aprendizado da rede

**Primeira Equação:** Ajusta os pesos entre a camada oculta e a de saída somando o gradiente proporcional à taxa de aprendizagem e à ativação transposta.

**Segunda Equação:** Atualiza os pesos entre a camada de entrada e a oculta utilizando os dados de entrada ( $X.T$ ) e o delta calculado.

**Objetivo:** Reduzir sistematicamente o erro global da rede a cada iteração.

```
# atualiza pesos oculta saída
self.pesos_oculta_saida += ( self.taxa * np.dot(self.saida_oculta.T, delta_saida))

# atualiza bias saída
self.bias_saida += (self.taxa * np.sum(delta_saida, axis=0, keepdims=True))

# atualiza pesos entrada oculta
self.pesos_entrada_oculta += (self.taxa * np.dot(X.T, delta_oculta))

# atualiza bias oculta
self.bias_oculta += (self.taxa * np.sum(delta_oculta, axis=0, keepdims=True))
```

## Slide 10 – Treinamento XOR (Conjunto de Dados)

### Definição das matrizes de treinamento

**Matriz X:** Contém as quatro combinações lógicas de **entrada bipolares/binárias possíveis**.

**Vetor y:** Contém os respectivos gabaritos esperados **para a operação lógica XOR**.

**Significado:** Representa a tabela verdade clássica da função XOR, um problema não separável por uma linha reta única.

```
6  # fixa semente aleatória
7  np.random.seed(42)
8
9  from src.mlp import mlp
10
11 # padrões XOR
12 X = np.array(
13     [
14         [0, 0],
15         [0, 1],
16         [1, 0],
17         [1, 1]
18     ]
19 )
20
21
22 # saídas desejadas
23 y = np.array(
24     [
25         [0],
26         [1],
27         [1],
28         [0]
29     ]
30 )
31
32
```

## Slide 11 – Cálculo do Erro Absoluto Total

### Mapeamento estatístico do aprendizado do modelo

**erro\_total:** Soma o **valor** absoluto de todas as diferenças individuais entre o esperado (**y**) e a **saída real da época atual**.

**Regra de Avaliação:** Quanto **menor** for esse indicador numérico, melhor e mais consolidado foi o aprendizado **alcançado** pela **rede**.

```
# calcula erro absoluto total
def calcular_erro(self, y):

    return np.sum(np.abs(y - self.saida_final))

# calcula erro quadrático médio
def calcular_mse(self, y):

    return np.mean((y - self.saida_final) ** 2)
```

## Slide 12 – Critério de Parada do Algoritmo

### Controle de término do laço infinito de épocas

A **verificação** condicional avalia se o erro acumulado atual é menor do que o limite estipulado pela tolerância.

Se a **condição** for verdadeira, interrompe (**break**) o treinamento imediatamente, significando que o modelo convergiu com sucesso.

```
RedeNeural-MLP > treinamento.py > ...
98
99     # verifica convergência
100     if erro_total < tolerancia:
101
102         convergiu = True
103
104         print()
105
106         print(f"Convergência atingida na época {epoca}")
107
108         print()
109
110         break
```

### Slide 13 - Treinamento da Rede MLP XOR - 1000 epocas

Iniciando treinamento...

Época 0 | Erro = 1.07397094  
Época 100 | Erro = 1.04617256  
Época 200 | Erro = 1.00720848  
Época 300 | Erro = 0.89294319  
Época 400 | Erro = 0.59597808  
Época 500 | Erro = 0.23896230  
Época 600 | Erro = 0.11074548  
Época 700 | Erro = 0.06573904  
Época 800 | Erro = 0.04510048  
Época 900 | Erro = 0.03373117

.....

Treinamento Finalizado

Status: Rede NÃO convergiu.

Erro Final: 0.02673263

Menor erro atingido: 0.02673263

Épocas utilizadas: 999

Resultados XOR

.....

|                |  |             |  |                  |
|----------------|--|-------------|--|------------------|
| Entrada: [0 0] |  | Esperado: 0 |  | Obtido: 0.070506 |
| Entrada: [0 1] |  | Esperado: 1 |  | Obtido: 0.919790 |
| Entrada: [1 0] |  | Esperado: 1 |  | Obtido: 0.924205 |
| Entrada: [1 1] |  | Esperado: 0 |  | Obtido: 0.096817 |

## Slide 13 – Gráfico de Evolução (Erro × Época)

### Análise visual e interpretação treinamento

**plt.plot(erros):** Desenha graficamente o comportamento do erro ao longo do ciclo de épocas.

#### Como Interpretar

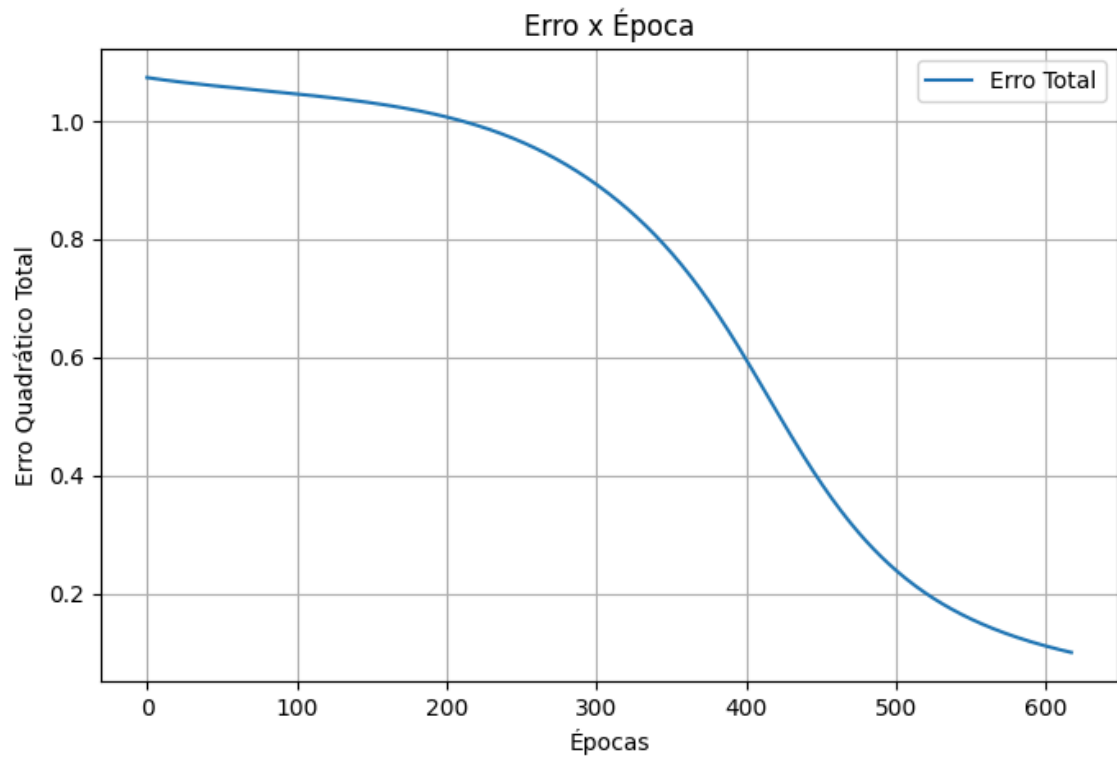
**Curva inclinada para baixo:** Representa o aprendizado ativo e o ajuste correto dos gradientes.

**Curva perfeitamente estabilizada (platô):** Indica convergência e fim da evolução útil.

```
158 | You, last week • inserindo e realizando o experimento A
159 | # gráfico erro x época
160 | plt.figure(figsize=(8, 5))
161 |
162 | plt.plot(erros, label="Erro Total")
163 |
164 | plt.title("Erro x Época")
165 |
166 | plt.xlabel("Épocas")
167 |
168 | plt.ylabel("Erro Quadrático Total")
169 |
170 | plt.legend()
171 |
172 | plt.grid(True)
173 |
174 | plt.show()
175 |
```

## Slide 13 Resultado - Gráfico de Evolução (Erro × Época)

- 1000 épocas de treinamento



## Slide 14 – Experimento A: Variação de Neurônios Ocultos

**Metodologia de teste para a camada intermediária:**

**Quantidades testadas:** Avaliação individual rodando com 2, 3, 4 e 5 neurônios ocultos.

**Métricas capturadas:** Quantidade total de épocas necessárias e erro final registrado.

**Objetivo:** Avaliar a influência direta da complexidade da camada oculta na eficiência do processamento.

```
35
36  # quantidades de neurônios ocultos
37  quantidades = [
38      2,
39      3,
40      4,
41      5
42  ]
43
44
```

## Slide 14 – Experimento A - Resultado

```
.....  
Experimento A  
.....
```

```
Testando 2 neurônios ocultos  
-----  
Épocas: 617  
Erro Final: 1.999832
```

```
Testando 3 neurônios ocultos  
-----  
Épocas: 617  
Erro Final: 1.995021
```

```
Testando 4 neurônios ocultos  
-----  
Épocas: 617  
Erro Final: 1.999491
```

```
Testando 5 neurônios ocultos  
-----  
Épocas: 617  
Erro Final: 1.990759
```

## Slide 15 – Experimento B: Variação da Taxa de Aprendizagem

### Análise da velocidade de atualização dos pesos

Hiperparâmetros testados: Execução com as taxas de **0.1**, **0.2**, **0.3**, **0.4** e **0.5**.

**Objetivo:** Medir a estabilidade e a velocidade de convergência, analisando o equilíbrio entre passos grandes (**risco de oscilação**) e passos pequenos (**lentidão**).

```
35
36 # taxas a testar
37 taxas = [
38
39     0.1,
40     0.2,
41     0.3,
42     0.4,
43     0.5
44 ]
45
```

## Slide 14 - Experimento B - Resultado

```
-----  
Experimento B  
  
Taxa de aprendizagem = 0.1  
-----  
Épocas: 617  
Erro Final: 1.997738  
  
Taxa de aprendizagem = 0.2  
-----  
Épocas: 617  
Erro Final: 1.986092  
  
Taxa de aprendizagem = 0.3  
-----  
Épocas: 617  
Erro Final: 1.804855  
  
Taxa de aprendizagem = 0.4  
-----  
Épocas: 617  
Erro Final: 1.567395  
  
Taxa de aprendizagem = 0.5  
-----  
Épocas: 617  
Erro Final: 1.987643
```

## Slide 16 – Experimento C: Influência da Inicialização (Sementes)

### Validação da estabilidade estocástica do modelo

Utilização sistemática de sementes como `np.random.seed(0)`, `seed(1)` e `seed(2)`.

**Explicação:** Cada semente força a geração de pesos iniciais completamente distintos.

**Objetivo:** Verificar a estabilidade da rede, comprovando se ela **converge de forma consistente** independente do ponto de partida.

```
37 # sementes
38 sementes = [
39     0,
40     1,
41     2,
42     3,
43     4
44 ]
45
46
47
48 for seed in sementes:
49
50     print()
51     print(f"Semente = {seed}")
52
53
54     # fixa aleatoriedade
55     np.random.seed(seed)
56
```

## Slide 17 – Experimento C - Resultado

```
.....  
Experimento C  
Influência dos Pesos Iniciais  
.....
```

```
Semente = 0  
Épocas: 617  
Erro Final: 1.998543
```

```
Semente = 1  
Épocas: 617  
Erro Final: 1.948505
```

```
Semente = 2  
Épocas: 617  
Erro Final: 1.936199
```

```
Semente = 3  
Épocas: 617  
Erro Final: 1.927552
```

```
Semente = 4  
Épocas: 617  
Erro Final: 1.785605
```

## Referencias

### **NumPy tutorials**

[https://numpy.org/numpy-tutorials/?utm\\_source](https://numpy.org/numpy-tutorials/?utm_source)

<https://medium.com/@alef.mattias/perceptron-multicamadas-do-zero-em-python-porta-l%C3%B3gica-xor-372a7291cf96>

### **Redes Neurais Explicadas: Resolvendo a Porta Lógica XOR com Retropropagação**

<https://youtu.be/xeSqeMPMb-0?si=Qs30roRLzYGENW4C>

### **Redes Neurais - A arquitetura da rede Multilayer Perceptron**

<https://youtu.be/qX7nL9CvKqw?si=PFPT3Wfp789Ev5wM>

### **Redes Neurais - Fundamentos e derivação do algoritmo de retropropagação**

[https://youtu.be/\\_Lj4DOv-4wQ?si=GAKUQJ6fHw6ccyU5](https://youtu.be/_Lj4DOv-4wQ?si=GAKUQJ6fHw6ccyU5)

### **Redes Neurais - Exemplos práticos de implementação e aplicação de redes MLP**

[https://youtu.be/L22X0SvhG1E?si=gBrD530\\_r5eujRyl](https://youtu.be/L22X0SvhG1E?si=gBrD530_r5eujRyl)

### **Redes Neurais - Ilustração da otimização da topologia de uma rede MLP**

[https://youtu.be/lfg0kp\\_wLw0?si=Y05hz3\\_4ajx9vwu3](https://youtu.be/lfg0kp_wLw0?si=Y05hz3_4ajx9vwu3)